

Part: I

Q1:

Ans:

DBMS	File System
Data stored in structured databases	Data stored in separate files
Minimizes the redundancy of data	High data duplication possible
Multiple users can access data simultaneously	Limited sharing among users
Ensures consistency using constraints and transactions	Difficult to maintain consistency across files

Now Suppose a university needs to manage:

Student records, Course registrations, Grades, Faculty information and Fee payments.

If a traditional file system is used, separate files might store student details, course enrollments, and grades. The same student information could appear in multiple files, leading to duplication and inconsistencies. For example, if a student's phone number changes, it would need to be updated in every file where it appears.

A DBMS is clearly the better choice because

1. Student information is stored once and referenced where needed.
2. Constraints ensure that grades cannot be assigned to non-existent students or courses.
3. The university can retrieve information easily.
4. Different users (students, faculty, administrators) can have different access permissions ensuring the secured access to the data.

Q2.)

Ans: In ER diagram weak entity is of Doubled rectangle while strong entity is denoted by single rectangle.

The primary key of a weak entity is formed by the combination of partial key and Primary key of strong key (owner key).

Q3)

Phone number should be modeled as a Multivalued attribute.

A simple attribute holds only one value. Since a student can have multiple phone numbers, this doesn't fit.

A multivalued attribute allows one student to have many phone numbers.

Q4)

Super Key: A super key is any set of attributes that can uniquely identify a tuple in a relation. It may contain extra unnecessary attributes.

Candidate Key: A candidate key is a minimal super key, meaning no attribute can be removed from it and still keep it unique. Every candidate key is a super key, but not every super key is a candidate key.

Candidate keys are roll_no and email.

Q5)

- Foreign Key Constraint is violated.
- It protects against: It ensures that every student_id in the Enrollment table belongs to a student who actually exists in the Student table.
- The insert operation should be rejected (or fail) until a valid student_id exists in the Student table.

Q6)

- Create a new relation/table (called a junction/bridge/associative table).
- The primary key consists of the combination of the primary keys of both participating entity tables (a composite key).
- Because in an M:N relationship, one record can be related to many records in the other table. A single column cannot efficiently store multiple values and would violate relational database design principles.

Part: II

Part A:

Entity and their attributes are:

- 1. Customer (customer_id)
- Restaurant (restaurant_id)
- Order (order_id)

- MenuItem (item_id)

Relationships and Cardinalities:

1. Customer → Order (1:N) : One customer can place many orders, but each order is placed by only one customer.
2. Restaurant → Order (1:N) : One restaurant can receive many orders, but each order is associated with exactly one restaurant.
3. Restaurant → MenuItem (1:N) : One restaurant can offer many menu items, but each menu item belongs to one restaurant.
4. Order → MenuItem (M:N) : One order can contain many menu items, and a menu item can appear in many different orders.

Part B:

Multivalued Attribute : Customer PhoneNumber

Because a customer may have multiple phone numbers.

Therefore, PhoneNumber can contain multiple values for a single customer.

Composite Attribute : Customer Address

Address can be divided into smaller components such as:

- House Number
- Street
- City
- State
- PIN Code

Therefore, Address is a composite attribute.

Part C:

Since Order and MenuItem have an M:N relationship, the junction table is below:

OrderMenuItem(OrderID, MenuItemID, Quantity)

- Primary Key: (OrderID, MenuItemID)
- Foreign Key 1: OrderID → Order(OrderID)
- Foreign Key 2: MenuItemID → MenuItem(MenuItemID)
- Additional Attribute: Quantity (stores the number of units of a menu item in an order)

Schema: OrderMenuItem(OrderID, MenuItemID, Quantity)

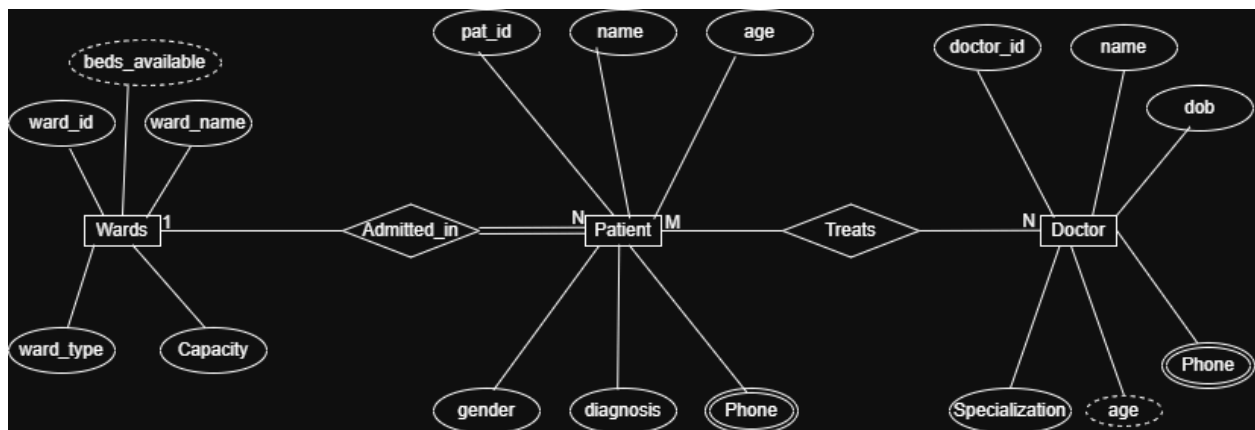
Final Schema:

OrderMenuItem(order_id, item_id, quantity)

- Primary Key: (order_id, item_id)
- Foreign Keys: order_id, item_id
- Additional Attribute: quantity

Part: III

Q1:



Q2:

The Doctor - Patient relationship is M:N because a single doctor can treat multiple patients during their shift, and at the same time, one patient can be attended to by several doctors throughout their hospital stay - for example, a general physician, a surgeon, and a physiotherapist might all treat the same patient. Since neither side is restricted to just one instance on the other, it is many-to-many. This creates an associative entity which can be called Treatment, and its composite primary key would consist of doctor_id and patient_id combined, since together they uniquely identify each doctor-patient pair.

Regarding participation: the patient side of the Admitted_In relationship has total participation, because every patient in this hospital system must be assigned to a ward - a patient floating in

the database without a ward makes no operational sense. The Ward side has partial participation since a ward can exist in the system even when currently empty. For the Treats relationship, both Doctor and Patient have partial participation -a doctor can be registered in the system before being assigned any patient, and similarly a patient may be logged before a doctor is formally assigned.

For attribute design, phone number was modelled as a multivalued attribute for both Doctor and Patient. The reason is that a person realistically has more than one contact number — a personal number, a family emergency contact, or a work extension. Storing it as a single simple attribute would lose that information, so a double ellipse was used to reflect that multiple values are possible for the same entity instance.

Submitted by:

Ankit Masoom (240140)